Exp No: 01

Date: 28/07/2026

Basic Networking Commands

1. arp -a

   Show IP-to-MAC address mappings [arp -a]

   O/P :- gateway (10.0.2.2) at 52:54:00:12:35:02 [ether] on enp0s3

2. hostname

   display the name of our computer

   O/P : localhost localdomain

3. Ipconfig /all

   Displays detailed network configuration

   O/P: windows IP configuration

   Host name . . . . : Indhu

   Primary DNS suffix . . . :

   Node type . . . . : Mixed

4. nbstat -a

   Troubleshoot NetBios name resolution

   O/P: Node IP Address : [192.168.1.100]

Name	Type	Status
Server 01	<00> unique	Registered
Server 01	<00> unique	Registered

5. netstat :-

Show network connection, ports, routes

O/P: Proto Recv-Q Send-Q local address State
tcp 0 0 192.168.1.100.22 Established

6. netstat -r :

Display the routing table of the computer

O/P: 21...00 ff eb da 62 66 ... TAP - Windows Adapter V9

1 ..... Software Loopback Interface 1.

7. nslookup

Performs DNS lookup

O/P: Default Server : unknown

Address : 127.0.0.1

8. Pathing :-

Combines ping and route analysis

O/P: 0 DESKTOP

1 192.168.1.1

9. Ping :

Test network connectivity

O/P : ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL]

10. Route

Displays & manipulates routing table

O/P: [-f] [-p] [-4 -6] Comand [destination]

Linux Networking Commands

1. ip

   Display ip address

   Configure network interface

   Add / delete ip address

   Managing routing tables

2. ip address add

   Assigns a new ip address to a network interfaces

   eg:- ip address add 192.168.1.254/24

   O/P : lo: <LOOPBACK UP, LOWER-UP>

3. Delete an ip address

   Remove the ip address from the interface

   eg: ip address del 192.168.1.254/24 dev enpos3

   O/P :- enpos3 : <BROADCAST, MULTICAST, UP, LOWER-UP>
   inet 10.0.2.15/24 brd 10.0.2.258 scope global
   dyramic enpos3

4. Bring Interface online

   Enables the network interface

   eg : ip set enpos3 up

   O/P enpos3 : mtu 1500 qdisc-fq-codel state

5. Interface online

   Displays the network interface

   eg : ip link set enpos3 down

   O/P : enpos3 : <BROADCAST, MULTICAST, DOWN, LOWER-UP>

6. Enable promiscous Mode

Allows the Interface receive all network packet

eg: ip link set enpos3 promisc on

o/p: enpos3: <BROADCAST, MULTICAST, PROMISC>

7. Add a default route:

Adds the default gateways for all network traffic

o/p: default via 192.168.1.254 dev enpos3

8. Add a route through gateway

Routes traffic for the 192.168.1.0/24 network through gateways 192.168.1.254

o/p: ip route add 192.168.1.0/24 dev eth0

9. Delete a route

Delete the route to the 192.168.1.0/24 network through gateway 192.168.1.0/24

of: ip route delete 192.168.1.0.24 via 192.168.1.254

10. ifconfig

configuring & Troubleshoot Network

o/p: Media State ... Media disconnected.

11. mtr

Combines the functionality of the ping and Traceroute Commands

o/p: Host

1. [illegible] Loss% 13% 8.1 10.2 0.1

0.0

12. a) mtr

b) mtr -g google.com [Show number ip address]

c) mtr -b google.com [numeric ip address & hostname]

d) mtr -c 10 google.com [numeric of ping that you]

13. tcp dump

Designed for capturing & displaying packets

i) tcpdump -i eth0 -c 10 host 8.8.8.8

Filter out traffic from a specific host

14. ping

Display best way to test connectivity b/t 2 nodes

O/P: minimum = 0 ms, maximum = 0 ms, Average = 0 ms

Result:-

The basic networking & Linux Commands are learned & executed successfully.

27/8/26

Exp.NO: 2

Date: 28/07/2026

Different type of Network Cables

Aim:-

To Study of different types of Network cables

1) UTP Cable
2) STP Cable
3) Coaxial Cable
4) Fibre optic Cable

Cable type	Category	Max data transmission	Adv / Dis	Application
UTP	Category 3	10 bps	Adv: Cheaper in cost	10 base-T Ethernet
	Category 5	up to 100 Mbps	Easy to install	fast ethernet, gigabit ethernet
	Category 5e	1 Gbps	dis: More prone to EMI	fast ethernet
STP	Category 6, 6a	10 Gbps	adv: Shielded, faster than UTP, less susceptible to noise	Gigabit Ethernet, 10 G Ethernet (55m) widely used in data centers
	Category 7	10 Gbps	Dis: expensive, greater installation cost	Gigabit ethernet, 10 G ethernet (...m)

Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	*High bandwidth * Immune to interferance * low loss bandwidth * versatile dis *limited distance * cost * size is bulky	speed by signal is 500m television network high speed internet connections.
fibre optic Cable	single mode multimode	1-100 Gbps	Adv * high speed * high bandwidth * high security * long distance dis * expensive * requires skilled installers.	* maximum distance of fibre optic cable is around 100m

Result:

The different types of network cables are learned and executed successfully.

4/8/26

Exp.NO: 3 CISCO PACKET TRACKER

Date: 04/05/24

Analyse the behaviour of network device using CISCO PACKET TRACKER simulator.

1. From the network component box, click & drag-and-drop the below components.

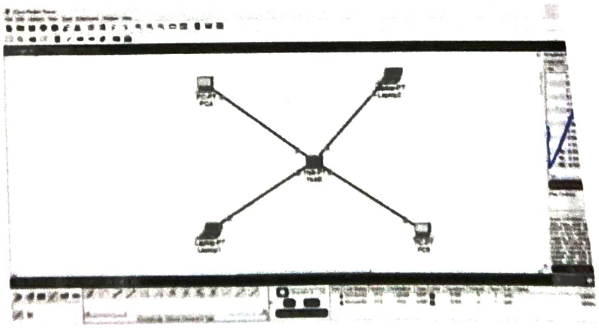
a. 4 Generic PCs and one HUB

2. click on connection

* Click on copper straight-through cable

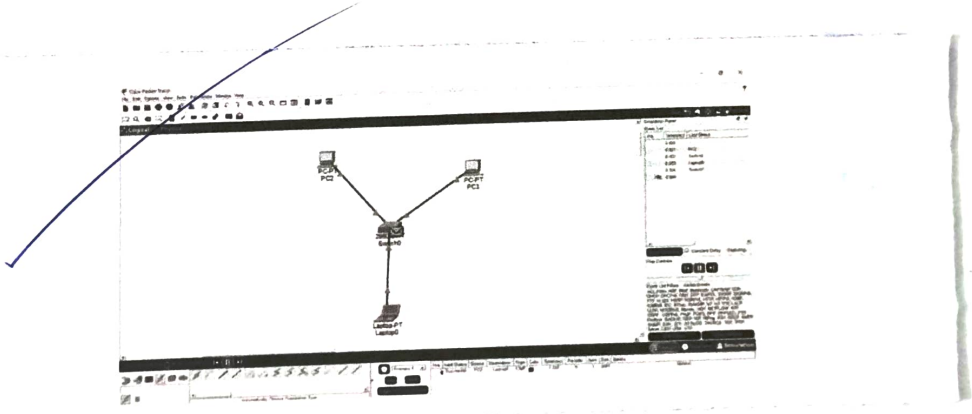
* select one of the pc and connect it to HUB using the cable

* similarly connect 2pcs to the HUB using copper straight-through cable.



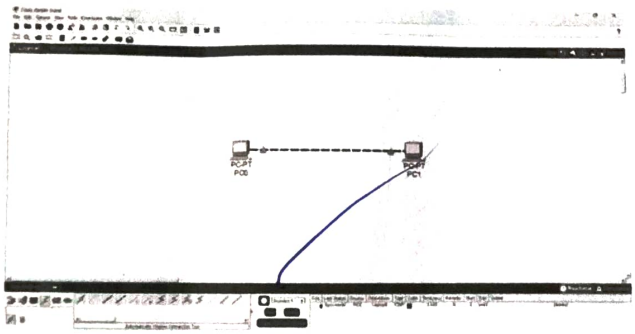
b. 4 Generic pcs and one switch

* click on copper straighe-through cable
* select one of the pc and connect it to switch using the cable
* similarly, connect 3pcs to the switch using copper straighe-through cable
* click on the pcs connected to switch go to the desktop tab, click on IP configuration, and enter an IP address
* set the message box to one pcs to another pc



c. PC to PC

* connect PC1 and PC2 using an Ethernet cable
* Assign ip address to both PCs in the same network
* Check the connection using ping between PC1 and PC2
* If the ping succeeds, the PC-to-PC connection is established.



Result:-

CISCO PACKER TRACER are learned and executed successfully.

Ex.NO:04 Setting up a Local Area Network

Date : 10/08/26

Aim:-

To design, setup and configure a local area network connecting four computers through a network, switch, assign static ip address to all device.

Diagram:

LAN was implement using STAR Topology

Implementation:-

Step 1: place the Device

Open the cisco packet tracker

From the device category panel, select end device and drag 4 PCs onto the workspace.

Step 2: cabel the Device

choose connection → Select copper. Straight through cabel.

click each pc, then switch and select an available fastEthernet port to complete to each connection.

Repeat for all four PCs green link lights Confirm a successfully physical connection.

Step 3: Assign ip address to PCs

Click a PC → Desktop tab → Configuration.

Enter the state ip address and submit mask as per the ip table for each pc.

Step 4 :-

click a Switch → CLI Tab

commands :-

Enable

configure Terminal

Interface vlan 1

IP address 10.1.1.5 255.0.0.0

no shutdown

exit.

Step 5 :-

Verify connectivity using ping

Check a pc → Desktop tab → command prompt

use the ping command to test connectivity to the other pc.

Step 6 :

Demonstrate data sharing

Add a device via its and connect it to the Switch.

Step 7 : observe packet flow in simulation mode

Switch to simulation mode.

Send a ping or web request bit two devices and use the play controls to step through the switch, as it travels hop by hop through the switch.

Result :

Setting up a Local Area network (LAN) we learned and executed successfully.

Exp.no:05
Date: 18/09/24

Experiments on Packet Capture tools: Wireshark

Capturing and analysing packets using wireshark tool:

To filter, capture, view, packets in wireshark tool. Capture 100 packets from Ethernet: IEEE802.3 LAN interface and save it.

Procedure:-

Select local area network connection in wireshark go to capture option

Select stop capture automatically after 100 packets

Save the packet

1. Create a filter to display only TCP/UDP packets. Inspect the packets and provide the flow graph.

Procedure

Select LAN connection is wireshark

Go to Capture options.

Selects Stop Capture automatically after 100 packets.

then Click the Start Capture

Save the packet

2. Create a filter to display only ARP packets and inspect the packets.

Procedure :-

Go to capture option.

Select Stop capture automatically after 100 packets

then click Start Capture

Search ARP packets in search bar.

Save the packets.

2. Create a file to display only DNS packets and provide the flow graph.

Procedure:-

Go to Capture option

Select Stop capture automatically after 100 packets

then click Start Capture

Search DNS packets in search bar;

TO see the flow graph click Statistics flow graph.

Result :-

Capturing and Analysing packet using wireless are studied and executed successfully.

29/9/25

Exno:06

Date: 07/09/26

Error Detection and Error Correction.

Aim:-

Write a program to implement error detection and correction using Hamming code concept, make a test run to input data stream and error correction features.

Sender:-

* Input to sender file should be a free of any length.
* Apply Hamming code concepts on the binary data and add redundant bits to it.

Code:

```
data=input("Enter a data:")
m=len(data)
r=0
while 2**r<(m+r+1)
    r+=1
hamming=[0]*(m+r+1)
j=0
for i in range(1,m+r+1):
if p(i-1)!=0
hamming[i]=int(data[j])
j+=1
for i in range(3):
    pos=2**i
    parity=0
for j in range(1,m+r+1):
    if j and pos!=0
        parity=hamming[j]
```

Receiver:-

Receiver program should read the input from channel life.

Apply hamming code on the binary data to check for error.

code:-

```
Receiver=input("Enter the received hamming code")
n=len(receiver)
hamming=[0]*(n+1)
hamming[:]=int(receiver[n-j])
r=0
while 2**r<n+1:
    r+=1
error=0
for q in range(r):
    pos=2**q
    parity=0
    for j in range(1,n+1):
        if j and pos!=0:
            parity=hamming[j]
            error+=parity^pos
if error!=0
    print("Error Detected",error)
else:
    print("There is no error")
```

```
for i in range(n,0,-1):
    print(hamming[i], end=" ")
print()
```

Sample Input:

Data bits: 1011

Hamming Code: 1100110

Receive detected at position: 6

Correct hamming code: 1100110

Result:-

Error detection and error correction using hamming code are learned and executed successfully

29/9/21

Exp No: 4
Date: 22/09/26

NMAP to Discover live hosts using Nmap scans on the TryHackMe platform.

Aim:-

To discover live hosts using Nmap scans (ARP, ICMP, TCP/UDP) on the TryHackMe platform room link, https://tryhackme.com/room/nmap1

Procedure:-

Task 1: Introduction

* ARP Scan: This scan uses ARP requests to discover live hosts.

* ICMP Scan: This scan uses ICMP requests to identify live hosts.

* TCP/UDP ping Scan: This scan sends packets to TCP ports and UDP ports to determine live hosts.

Task 2: Subnetworks:-

Network Segment - It is a collection of computers linked together through a common "medium".

Subnetwork - It is usually the equivalent of one or more network segments linked together and configured to utilize the same router in an IP network.

* Subnets with /16

* Subnets with /24

Task 3: Enumerating Targets

We need to identify the targets we want to scan before we conduct any "action"

1) list

- < Machine-IPs > scanme.nmap.org < example.org >

- will scan 3 IP addresses.

2) range

- 10.11.12.15-20

- will scan 6 IP addresses.

3) subnet

- <MACHINE-IP>/30

- will scan 4 IP addresses

nmap -sL <TARGETS>

Task 4: Discovering live hosts:-

1) ARP from link layer - commonly used for man-in-middle - Attacks (ARP spooting and ARP poisoning)

2) ICMP from network layer - commonly used DDos attacks

3) TCP from Transport layer - SYN flooding TCP reset, TCP session hijacking attacks.

Task 5: Nmap Host Discovery using ARP

1) when a privileged user has to scan LAN (Local Area Network), Nmap uses ARP requests.

2) to discover ethernet hosts without port scanning - live systems use nmap -sn <Targets>

3) nmap -PR -sn <TARGETS>

-PR specifies that you only want an ARP scan

4) nmap -PR -sn <MACHINE IP>/24

Task 6: Nmap host discovery using ICMP

1) Add -PE option to utilize ICMP echo requests to discover live hosts

2) Nmap -P -sn TARGET to check host is live

3) -PP option instructs Nmap to use ICMP timestamp requests.

nmap -PP -sn TARGET

4) -PM option can enable this scan to ICMP address mask requests.

nmap -PM -sn TARGET.

Task 7: Nmap host discovery using TCP and UDP.

1) Nmap to use TCP SYN ping use -PS followed by the port number, range, list or a combination of this options.

-PS21, -PS25 targets ports 21, 22, 23, 24, 25

2) nmap -PS -sn TARGET check the TCP port is open.

3) TCP ACK ping - you must be running Nmap as a privileged user.

4) nmap -PA -sn TARGET to discover online hosts on the target's subnet

Task 8: Using Reverse-DNS lookup

1) Nmap is to use reverse-DNS Internet hosts

2) -R option to query the DNS server even for offline hosts.

Task 9: Summary.

ARP scan, ICMP echo scan, ICMP timestamp scan, ICMP address mask scan, TCP SYN ping scan, TCP ACK ping scan, UDP ping scan.

Result:

NMAP to discover live host using tryhackme platform was successfully executed.

Exp. 8

Date 28/09/26

End to End communication at Transport layer.

Aim:-

Implement echo client server using TCP / UDP packets.

Procedure:-

* Create a socket using python socket programming
* Specify the IP address and port number.
* Bind the server socket to the IP address and port.
* Make the server listen for client connections.
* Connect the client to the server.
* Send and receive messages between client and server.
* Display the received message.
* close the socket connection.

Code:-

TCP - Server:-

```
import socket:-
server = socket.socket(socket.AF_INET, socket.
                       SOCK_STREAM)
server.bind("127.0.0.1", 5000)
server.listen(1)
print("Server is waiting for connection...")
conn, addr = server.accept()
print("Client connected")
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("Client:", data)
    conn.send(data.encode())
conn.close()
server.close()
```

TCP - receiver (client)

```
import socket
client = socket.socket(socket.AF_INET, socket.
                       SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter message: ")
client.send(message.encode())
reply = client.recv(1024).decode()
print("Server:", reply)
client.close()
```

Terminal 1 - Server

```
Python tcp-server.py
Server is waiting for
connection....
client connected
Client: Hello
```

Terminal 2

```
Python tcp-client.py
Enter message: Hello
Server: Hello.
TCP Echo is working
```

Result:

The TCP/UDP packets using Echo client Server socket program is executed successfully.

b)

Aim:-

Implement chat client server using Tcp, packets.

code:-

chat - sever side

```
import socket
sever = socket.socket(socket.AF_INET, socket.SOCKET_STREAM)
sever.bind("127.0.0.1", 6000)
sever.listen(1)
print("waiting for client....")
conn, addr = sever.accept()
print("client connected")
while True:
    message = conn.recv(1024).decode()
    if message.lower() == "bye":
    print("client ended chat")
    break.
    print("client:", message)
    reply = input("sever:")
    conn.send(reply.encode())
    if reply.lower() == "bye"
conn.close()
sever.close()
```

chat - client side

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 6000))
print("connected to server")
while True:
    message = input("client: ")
    client.send(message.encode())
    if (message.lower() == "bye": break
    reply = client.recv(1024).decode
    if reply.lower() = "bye":
        break
    client.close()
```

Terminal 1:

```
python chat_server.py
waiting for client....
client connected
client: Hello
server: Hi
client: How are you?
server: fine
```

Terminal 2:

```
python chat_client.py
connected to server...
client: Hello
server: Hi
client: How are you?
server: Fine.
```

Result:-

The chat programming using sockets by TCP client & server was implemented successfully.

29/9/22

Exno: 9

Date: 29/09/21

Sliding window protocol

Aim:-

Write a program to implement flow control at data link layer using sliding window protocol simulate the flows from one node to another.

code:-

```
import time
window = int(input("Enter window size:"))
message = input("Enter message:")
start=0
while start < len(message)
print ("SENDER")
end = start+window
if end > len(message):
    end = len(message)
# Sender buffer.
Sender_buffer = [ ]
for i in range(start, end):
    frame = (i, message[i])
    Sender_buffer.append(frame)
    Print ("Frame NO:", i, "DATA:", message[i])
Print ("\n Sender Buffer:")
Print (Sender_buffer)
# Delay
Print ("\n waiting for receiver...")
time.sleep(1)
```

```
# Receiver
print("RECEIVER")
expected = start
error = false
for frame in sender_buffer:
frame_no = frame[0]
data = frame[1]
Print("Received frame:", frame_no, "DATA:", data)
if frame_no == expected:
    expected = expected + 1
else:
    error = True
    break
# Receiver buffer
if error:
    receiver_buffer = ("NACK", expected)
    Print("In Receiver Buffer:")
    print(receiver_buffer)
else:
    ack = expected - 1
    receiver_buffer = ("Ack", ack)
    Print("In Receiver_buffer")
# Sender reads ACK [NACK
Print("In SENDER CHECKING RESPONSE")
response = receiver_buffer[0]
number = receiver_buffer[1]
```

```
if response == "ACK":
    Print("ACK received :", number)
    start = number+1
else:
    PRint ("NACK received:", number)
    start = number
Print ("TRANSMISSION COMPLETED")
```

Output:

Enter window size: 3

Enter message : Hello

SENDER

Frame NO : 0 DATA : H

Frame NO : 1 DATA : e

Frame NO : 2 DATA : l

Sender buffer:

[(0,'H'), (1,'e'), (2,'l')]

waiting for receive

RECEIVER

Received frame: 0 DATA : H

Received Frame : 1 DATA : e

Received Frame : 2 DATA : l

Receiver Buffer : ('Ack', 2)'

Sender checking response

Ack Received : 2

SENDER

Frame NO : 3 DATA : l

Frame no : 4 Data : o

Sender Buffer:

[(3,'l'),(4,'o')]

waiting for receive.

RECEIVER

Received frame : 3 DATA : l

Received frame : 4 DATA : o

Receiver Buffer : ('Ack', 4)

Sender, Checking Response

Ack received : 4

Result:-

Sliding window protocols are learned and executed Successfully.

Exp.No: 10

Date: 5/10/26

Subnetting in cisco packet tracer

Aim:-

To implement classless IP subnetting using CISO packet tracer and verify the connectivity between different subnets.

Requirements:-

CISCO packet Tracer

Routers

Switches

PCs

Ethernet Cables.

Procedure:

* Open CISCO packet tracer
* Create a new network topology
* Add routers, switches and pcs to the workspace connect the device using ethernet cables.
* Divide the network into smaller subnets using the /27 subnet mask
* Configure the IP address, subnet mask and default gateway for each PC.
* Configure the required switch ports.
* Enable the router interfaces using the no shutdown command.

* check the ip configuration of all devices use the ping command to check connectivity between pcs

* verify that the devices on the network communicate successfully.

IP Addressing :-

	IP address	Subnet Mask	Default gateway
PC-PT-PC0	192.168.2.12	255.255.255.192	192.168.3.1
PC-PT-PC1	192.168.1.12	255.255.255.192	192.168.3.1
PC-PT-PC2	192.168.1.12	255.255.255.192	192.168.1.1
PC-PT-PC3	192.168.1.11	255.255.255.192	192.168.1.1

Result :-

Thus, Classless IP Subnetting was successfully implemented using cisco packet Tracer and. Connectivity between the network devices was verified successfully.

Exp.No:11 PING PROGRAM

Date: 6/10/25

Aim:-

To implement a ping program using socket programming to check the connectivity between two network devices.

Program:-

```
import socket
import time
import sys
def udp_ping_server(host="127.0.0.1", port=12000):
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_socket.bind((host,port))
print(f"[Server] listening on {host}:{port}")
while True:
    msg, client_address = server_socket.recvfrom(1024)
    print(f"[Server] Received '{msg.decode()}' from
                {client_address}")
    server_socket.sendto(msg, client_address)
def udp_ping_client(server_host="127.0.0.1", server_port
    =12000, count=5):
    client_socket.settimeout(1)
    for i in range(1, count+1):
```

```
msg = f"ping {i} {time.time()}"
start = time.time()
client_socket.sendto(
    msg.encode(), (server_host, server_port)
try:
    data, _ = client_socket.recvfrom(1024)
    end = time.time()
    rtt = (end - start) * 1000
    print(
        f"Reply from {server_host}: {server_port}" f" {data.decode()}
        RTT = {rtt:.2f} ms")
except socket.timeout:
    print(f"Request {i} timed out")
client_socket.close()
if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "server":
        udp_ping_server()
    else:
        udp_ping_client()
```

Output:-

```
Python udp_ping.py server
[SERVER] listening on 127.0.0.1:12000
[SERVER] Received 'ping 1779147648.6564088' from
('127.0.0.1', 55522)

Python udp_ping.py
```

Reply from 127.0.0.1:12000 |Ping| 1791476483.6stew

=0.99 ms

Reply from 127.0.0.1:12000 | Ping 2 1791476483.

6980255| RTT =0.25 ms

```
Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=55ms TTL=125
Reply from 20.0.0.2: bytes=32 time=56ms TTL=125
Reply from 20.0.0.2: bytes=32 time=60ms TTL=125
Reply from 20.0.0.2: bytes=32 time=55ms TTL=125

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 55ms, Maximum = 60ms, Average = 56ms

C:\>tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2   11 ms      6 ms      5 ms      192.168.1.250
  3   29 ms     26 ms     25 ms      192.168.1.245
  4   22 ms     28 ms     20 ms      20.0.0.2

Trace complete.
```

Result :-

Thus, the ping program is executed successfully.

Exp.No:12

Date: 6/10/24

Implementation of Subnetting in Cisco packet Racer Simulator

Aim:-

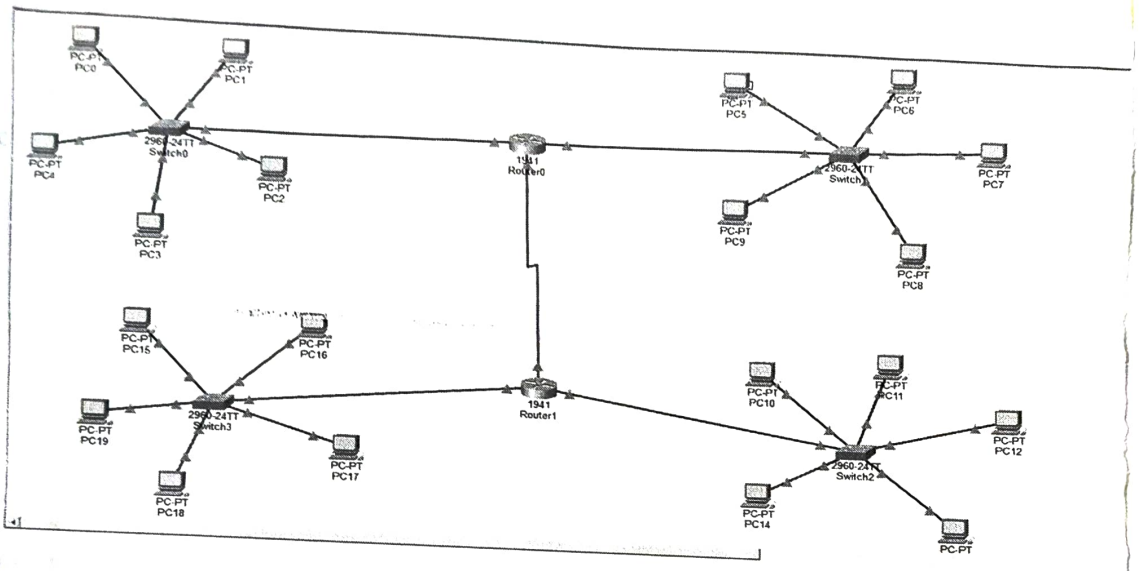
To Implement the Subnetting in Cisco packet Tracer Simulator.

Procedure:-

* Open cisco packet tracer and create new network topology.
* add the required routers, switches and PCs to the workspace
* Connect the device using appropriate coppes cables.
* use the network 192.168.1.0/24 and divide it into 12 networks.
* Configure the router interfaces with the required IP addresses and Subnet mask 255.155.255.224
* use enable and configure terminal in the router CLI.
* Enter the interface configuration terminal in the router and assign the ip using
  i) Ip address <IP address> 255.255.255.224
  ii) no shoutdown
* configure the required switch ports in access mode
* Assign each PC its IP address subnet mask and default gateway.

* use ping command to test connectivity b/t pcs and the router

* If ping is successful, the network is configured correctly.



Result :-

The network was successfully subnetted using the 127 subnet mask in CISCO packet. Process connectivity between the devices was successfully verified using ping command.

Exp.No:13

Date:12/10/26

Routing at network layer

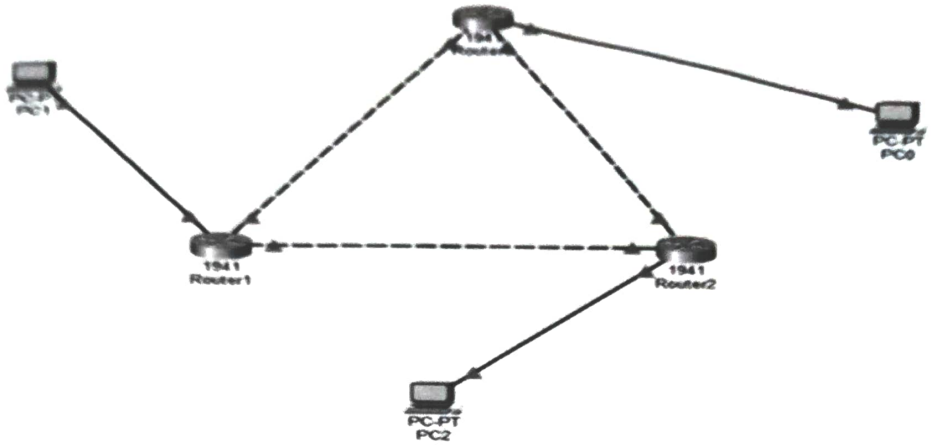
a) Simulate static routing protocol configuration using CISO packet Tracer

Aim:-

TO simulate a static routing configuration using ciso packet tracer.

Procedure:-

* Open cisco packet tracer and create the given network topology
* connect 3 routers and pcs using suitable cables
* Enable all router interfaces using no shoutdown
* Configure static routes on router0, router1, router2 using ip mobile
* Set main and backup routes using different administrative distance value test connectivity.
* use tracer to verify the routing path.
* Disconnect the main link and verify that the backup route works
* Thus, a static routing is successfully configured and verified.



Result:-

Thus, static routing was successfully configured in cisco packet tracer, connectivity between all networks was verified successfully using ping.

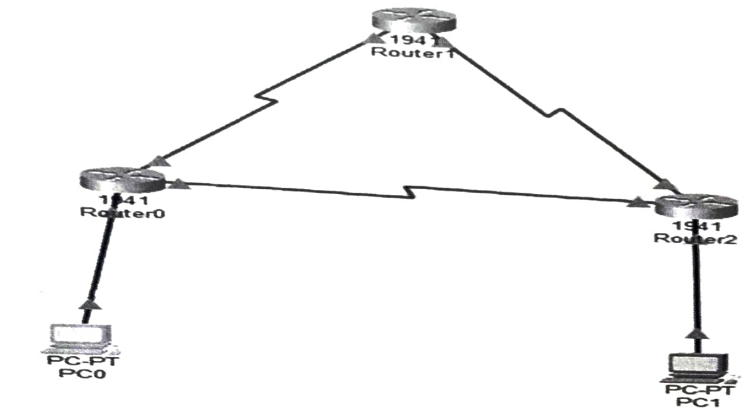
b. Simulate RIP using CISCO packet Tracer.

Aim:-

To Simulate RIP using CISCO packet Tracer

Procedure:-

* Create the topology with routers and 2PCs
* connect the device using Suitable Cables.
* Assign the given IP address to both PCs
* Configure fastethernet and Serial interface of all routers.
* Enable Interface using no shoutdown
* Set clock rate & 4000 and bandwidth 64 on DCE Serial Interface
* Enable RIP on all routers using router rip
* Add directly connected networks using the network command
* test connectivity using ping 20.0.0.2
* use tracer 20.0.0.2 to verify the selected route
* Disconnect the direct router 0 - router 2 link and use tracer again to verify RIP selects the alternate route.



Result:-

RIP successfully configured and dynamic routing between pc0 and pc1 was verified.

Exp no:4

Date: 12/10/24

Interworking with routers in CISCO PACKET TRACER Simulator.

a) Design and configure a simple interwork using router.

Aim:-

To design and configure a simple interwork using a router in Cisco packet tracer and fast connectivity bit two pcs.

Procedure:-

Step 1: configuring router:

* Select the router and open CLI
* Press enter to start configuring router.
* type enable to activate the previledge mode.

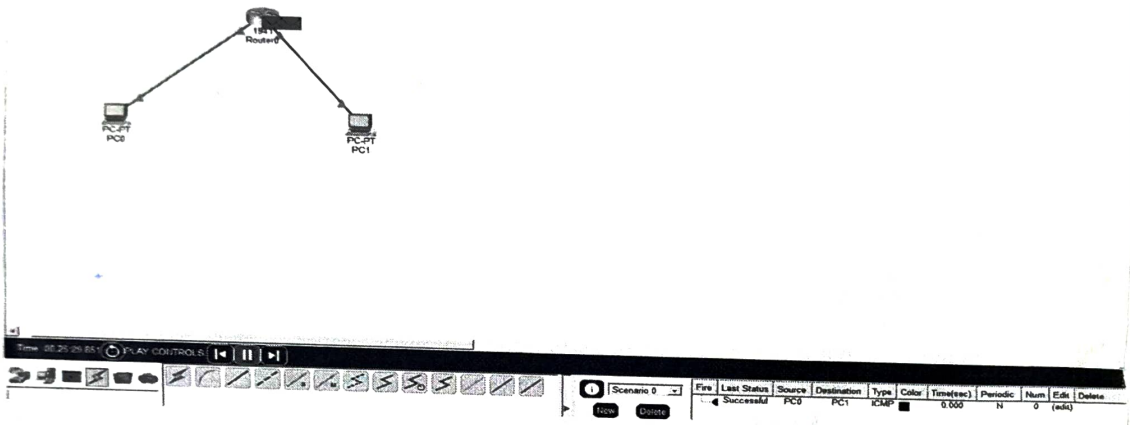
Step 2:

* Assign ip address to every pcs in the network
* Select the PC, Go to the desktop and select Ip configuration and assign an IP address default gateways, subnet mask.
* Assign the default gateway of PC0 as 192.168.10.1
* Assign the default gateway of PC1 as 192.168.10.2

Step 3:-

* Connect FastEthernet 0 port of PC0 with fast ethernet 0/0 port of router1 using a copper straight through cable.

* Connect the fast Ethernet 0 port of PC1 with fast ethernet 0/1 port of router 1 using a copper straight through cable.



Result:-

The simple internetwork was successfully designed and configured using a router. Connectivity between pc0 and PC1 was successfully verified using PDU

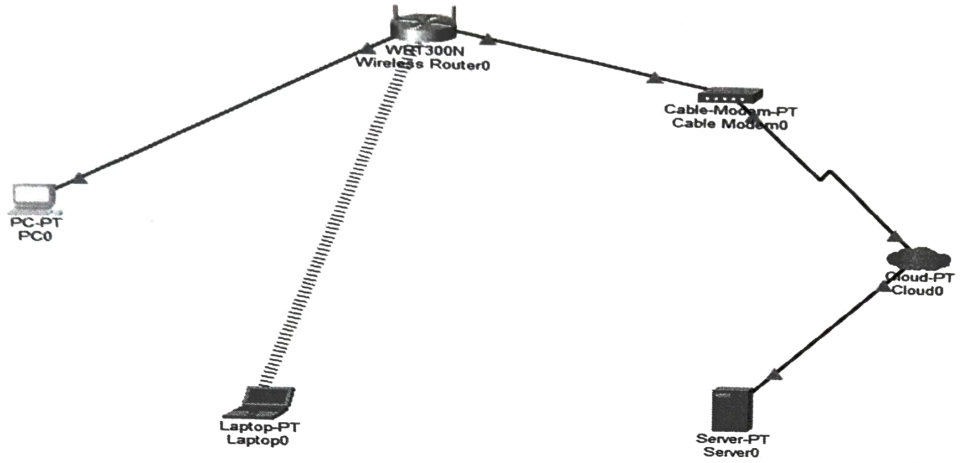
b) Design and configure an internetwork using wireless router DHCP server and internet cloud.

Aim:-

To Design and configure an internetwork using a wireless router, DHCP server and internet cloud in cisco packet tracer.

Procedure:-

* Open cisco packet tracer and create the required network topology
* Connect PC, laptop, wireless router, cable modem, internet cloud and server.
* Configure the wireless router with SSID homenetwork and enable DHCP.
* Connect the laptop to the wireless network and set the PC to DHCP.
* Configure the server with IP 208.64.220.220 and enable DHCP and DNS
* Verify connectivity using ip config and ping cisco.com



Result:-

The internetwork was successfully designed and configured using a wireless router, DHCP server and internet cloud and connectivity was successfully verified using ping cisco.com.

6) Design and configure an internetwork in your lab using switch, router and Ethernet cables

Aim:-

To design and configure an internetwork using a switch, router and ethernet cables and verify connectivity bit computers.

Procedure:-

* Open cisco packet tracer and place computer, a switch and router in the workspace.
* Connect the computers to the switch using ethernet cables.
* Connect the switch to the router using an ethernet cable.
* Assign static ip addresses, subnet masks and default gateway addresses to the computers.
* Configure the router interfaces with appropriate ip address and enable the interface
* Observe the successful transmission of packets.

Result:-

Thus, an internetwork was successfully designed and configured using switch, router and ethernet cables in Cisco packet tracer.

Exp.No:15

Date: 13/10/25

Analyze various types of Server using Webalizer tools

Aim :-

To analyze web server log files using the webalizer tool and study server traffic statistics

Procedure :-

* Install and configure the webalizer tool on a system.
* Obtain the web server log file, such as an apache access log
* Configure webalizer to read the server log file.
* Execute the webalizer command to generate statistical reports.
* Open the generated HTML report in a web browser
* Analyze the number of visitors, requests page view hits and bandwidth usage.
* Compare server activity using the generated reports

Result :-

Thus, web server log files were analyzed successfully using the webalizer tool and server traffic statistic were studied.